AEGIS Backend + AI Next Steps

Roadmap from local MVP to integrated Android product, analyst workflow, real AI, and production hardening.

Current baseline

- FastAPI backend with SQLite local database and raw JSON payload storage.
- Schema validation, body-token auth, idempotent ingestion, and processing status tracking.
- Normalization for device posture, app inventory, and redacted important logs.
- Deterministic risk scoring plus audited AI/LLM stub for high-risk evidence review.
- Analyst feedback endpoint and Docker Compose path for production-style services.

1. Android

Connect real agent uploads to POST /api/v1/telemetry and verify end-to-end processing.

2. Dashboard

Add a minimal analyst UI for latest risk, timeline, payload summary, logs, and feedback.

3. Rules

Tune deterministic scoring using real safe, suspicious, and critical device examples.

4. Real AI

Add a provider behind the LLMAnalyzer interface with strict JSON and evidence references.

5. Harden

Move to Postgres, Redis queue, migrations, stronger auth, metrics, and retention.

Guiding principle

- Keep backend validation, storage, and risk state deterministic.
- Use AI to explain suspicious evidence and assist analysts, not to replace system truth.
- Add production infrastructure only after integration behavior is stable.

Phase 1 And 2

First connect the real Android data path, then give analysts a simple place to inspect it.

Phase 1 - Connect Android Agent

Configure the agent backend URL. Send real scans to POST /api/v1/telemetry. Confirm 202 responses, duplicate handling, raw JSON files, normalized rows, and latest risk lookup. The exit goal is one real device completing scan, upload, processing, and risk review.

- Use the current Android payload unchanged.
- Keep enrollment_token body auth for compatibility.
- Verify raw payloads before debugging worker behavior.
- Test both clean and suspicious devices.
- Save sample payloads as fixtures for backend tests.

Phase 2 - Minimal Analyst Dashboard

Build a small operational dashboard over the existing API. It should show device risk, timeline, payload summaries, important logs, and analyst feedback. Avoid heavy product features until the evidence flow feels right.

- Device list sorted by latest risk.
- Device detail page with timeline records.
- Payload detail with normalized summary.
- Redacted logs with message hash references.
- Feedback form for true positive, false positive, benign, or needs more data.

Exit criteria A real device can upload telemetry, and an analyst can review the resulting risk in the UI.

Phase 3 And 4

Improve deterministic rules first, then connect real LLM providers through the existing safe interface.

Phase 3 - Improve Rules With Real Data

Use safe, suspicious, and critical telemetry from real devices to tune rule scoring. Rules should stay explainable and covered by tests. This gives the AI layer better evidence and keeps the product reliable.

- Create fixtures from real uploaded payloads.
- Tune scores for root, integrity failure, patch age, bootloader state, and sideloaded sensitive apps.
- Add reasons that are readable by analysts.
- Track obvious false positives and adjust thresholds.
- Keep expected risk outputs in tests.

Phase 4 - Add Real AI Provider

Keep the LLMAnalyzer interface. Add provider settings, timeout policy, structured JSON enforcement, redaction checks, and evidence-reference validation. Model output is advisory and audited.

- Call AI only for suspicious or high-risk cases.
- Send redacted evidence bundles, never raw secrets.
- Reject findings without evidence references.
- Store every attempt in ai_model_runs.
- Compare provider outputs against analyst_feedback later.

Exit criteria Risk labels are stable, and high-risk cases can produce audited AI findings safely.

Phase 5 - Production Hardening

Move from local MVP behavior to a deployable service without changing the core data contracts.

Data Layer

Make Postgres the production default, add Alembic migrations, define retention, and document backup/restore.

Queue + Worker

Use Redis/RQ, Celery, or equivalent. API accepts payloads quickly; worker handles normalization, scoring, and AI runs.

Auth + Integrity

Move from body token to bearer auth, then mTLS later. Add Play Integrity backend challenge/decode flow.

Observability

Add request logs, processing metrics, failed payload alerts, model-run errors, and dashboard health checks.

Security

Protect raw telemetry, enforce redaction before AI calls, rotate secrets, and separate dev/prod settings.

Release Path

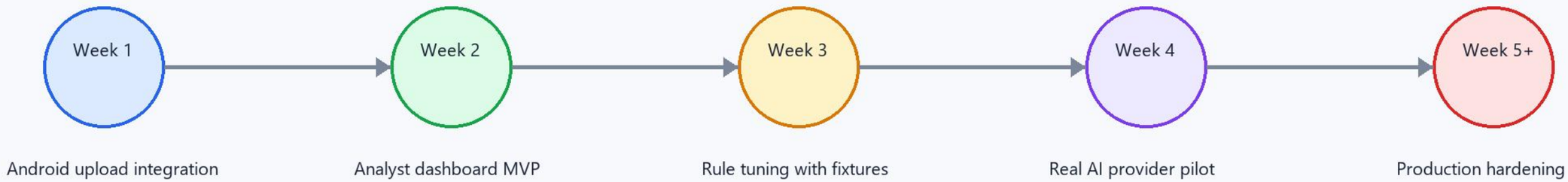
Use Docker Compose first, then cloud deployment after API, worker, database, and queue behavior is stable.

Production readiness checklist

- Postgres migrations run cleanly from an empty database.
- Worker retries failed payloads safely and marks permanent failures clearly.
- Auth no longer depends on body enrollment_token for production clients.
- AI provider can be disabled without breaking risk scoring.
- Backups and telemetry retention are documented before real user data is stored.

Suggested Execution Order

Small steps, each with a concrete demo result before moving deeper into infrastructure.



Do next, immediately

- Pick one Android device and point uploads to http://127.0.0.1:8080/api/v1/telemetry.
- Capture one clean payload and one suspicious/high-risk payload.
- Turn those payloads into backend test fixtures.
- Build the first dashboard screen: device list with latest risk.
- Use feedback labels to measure rule quality before trusting real AI output.

Decision checkpoint

After the dashboard can review real device telemetry, decide whether to invest next in rule quality, real AI, or production deployment.